argument, which you dictate to Ninja through the use of brace brackets, similar to an f-string. The following listing shows how to tie a named value in the URL with the view function's arguments.

Listing 11.6 An endpoint to fetch a single `Promoter` object by its ID

```
# RiffMates/promoters/api.py
from django.shortcuts import get_object_or_404
...
@router.get("/promoter/{promoter_id}/",
    response=PromoterSchema)
def promoter(request, promoter_id):
    promoter = get_object_or_404(Promoter, id=promoter_id)
    return promoter
```

Denote URL arguments, using brace brackets.

This endpoint returns a single object and uses the PromoterSchema to serialize it.

Match the named URL argument with an argument to the endpoint view function.

Like in a page view function, use get_object_or_404() to find the requested object, and then return it.

To dive deeper into arguments and serialization, let's move on from promoters and start building a more complex set of API endpoints. The goal is to provide a full set of CRUD operations on the `Venue` model, including the ability to show a subset of venues at once through the application of a filter. Again, if this was for a mobile application, you'd start with the venue listing screen, but this time, you'd also have an input field that allowed you to only show those venues that match the name in the field.

Like with the other RiffMates apps, you need a new api.py file, this time in the `bands` app directory. To output a `Venue` object, you need a serializer, which is also based on a `ModelSchema` class. Earlier, I hinted that you can declare your own fields on a `Schema`. You can do this by building a serializer that is independent of a model, or you can augment a `ModelSchema` by adding fields. I'll show you the latter by defining the `VenueSchema` serializer.

I want to add two fields that aren't in the `Venue` model to the `VenueSchema`, both of which can be helpful for developers consuming your API. A *slug* is a term from the newspaper industry, meaning a short name for an article. Slugs sometimes get used on the web inside of URLs. Consider the readability difference between `/song/43/` and `/song/paint-it-black/`; the latter is human readable and gives search engines a clue regarding what the URL points to. A common technique is to combine the two ideas: `/song/paint-it-black-43/` keeps the human-readable content, while making it easy to parse the song's ID off the end of the URL. An integer-based primary key is typically significantly faster to find in a database than searching on a string. With the mixed technique, you don't even need to store the slug, as the `43` part is enough to do your look-up.

The second field to add to the serializer is a URL for the object's GET method. This provides a shortcut for developers using the listing GET and wanting to fetch more information on a specific object. If you've designed your API well, the URL will be the same for PATCH and DELETE, meaning the programmer doesn't have to know how to construct your API endpoint—they can just use the information in the serialized object.